MINI PROJECT

EX. NO: 14 CAMPUS PLACEMENT AND RECRUITMENT MANAGEMENT

DATE: 04.10.2026

Aim

To develop a Campus Placement and Recruitment Management system using SQL and PL/SQL to store, manage, and analyze student eligibility, company offers, department-wise placement, and salary packages.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Campus Placement and Recruitment Management system is developed to examine student eligibility, company offers, department-wise placement, and salary packages. The system stores the registration number, student name, department, CGPA, number of backlogs, company, offer amount (in LPA), and placement status of each student.

SQL queries are used to retrieve and analyze placement information, while PL/SQL functions and procedures are used to determine placement eligibility based on a specified CGPA threshold, update a student's placement status and offer details, generate a department-wise placement report with placement counts and salary statistics, and handle exceptions.

Dataset Source

- Dataset: Campus Placement sample dataset
- Table: PLACEMENT_RECORDS
- Records used: 25 (5 departments: CSE, IT, ECE, EEE, MECH; 5 students each)
- Added field: STUDENT_ID (record number 1 to 25 used as the primary key)
- Offer amounts are in LPA (lakhs per annum) and are rounded to 2 decimal places to match the NUMBER(5,2) data type; students who are not placed have NULL company and offer amount.
- Eligibility rule: a student is eligible when CGPA >= the specified threshold (7.5 is used in the function test); a minimum CGPA of 6.5 is required to update a placement offer.

Database Table

PLACEMENT_RECORDS

Column Name	Data Type	Constraint
STUDENT_ID	NUMBER(4)	PRIMARY KEY
REG_NO	VARCHAR2(15)	NOT NULL
STUDENT_NAME	VARCHAR2(30)	NOT NULL
DEPARTMENT	VARCHAR2(10)	NOT NULL
CGPA	NUMBER(4,2)	NOT NULL, 0 to 10
BACKLOGS	NUMBER(1)	>= 0
COMPANY	VARCHAR2(20)	NULL if not placed
OFFER_AMOUNT	NUMBER(5,2)	> 0, NULL if not placed
PLACEMENT_STATUS	VARCHAR2(12)	NOT NULL, Placed or Not Placed
(REG_NO)	---	UNIQUE

TABLE CREATION

Oracle

```
CREATE TABLE placement_records (
   student_id NUMBER(4) PRIMARY KEY,
   reg_no VARCHAR2(15) NOT NULL,
   student_name VARCHAR2(30) NOT NULL,
   department VARCHAR2(10) NOT NULL,
   cgpa NUMBER(4,2) NOT NULL,
   backlogs NUMBER(1),
   company VARCHAR2(20),
   offer_amount NUMBER(5,2),
   placement_status VARCHAR2(12) NOT NULL,
   CONSTRAINT chk_cgpa CHECK (cgpa BETWEEN 0 AND 10),
   CONSTRAINT chk_backlogs CHECK (backlogs >= 0),
   CONSTRAINT chk_status CHECK (placement_status IN ('Placed', 'Not Placed')),
   CONSTRAINT chk_offer CHECK (
   (placement_status = 'Placed' AND company IS NOT NULL AND offer_amount > 0)
   OR (placement_status = 'Not Placed' AND company IS NULL AND offer_amount IS
   NULL)),
   CONSTRAINT uq_reg_no UNIQUE (reg_no)
);
```

SQLite

```
CREATE TABLE placement_records (
   student_id INTEGER PRIMARY KEY,
   reg_no TEXT NOT NULL,
   student_name TEXT NOT NULL,
   department TEXT NOT NULL,
   cgpa REAL NOT NULL CHECK (cgpa BETWEEN 0 AND 10),
   backlogs INTEGER CHECK (backlogs >= 0),
   company TEXT,
   offer_amount REAL,
   placement_status TEXT NOT NULL CHECK (placement_status IN ('Placed', 'Not
   Placed')),
   CHECK ((placement_status = 'Placed' AND company IS NOT NULL AND
   offer_amount > 0)
   OR (placement_status = 'Not Placed' AND company IS NULL AND offer_amount IS
   NULL)),
   UNIQUE (reg_no)
);
```

Table Created

The PLACEMENT_RECORDS table was created successfully with primary key, unique, CHECK, and NOT NULL constraints.

DATASET INSERTION

```
INSERT INTO placement_records VALUES (1, '20524UCSC001', 'Aarav Kumar', 'CSE',
9.1, 0, 'Zoho', 9.5, 'Placed');
INSERT INTO placement_records VALUES (2, '20524UCSC002', 'Divya Lakshmi',
'CSE', 8.7, 0, 'TCS', 7.0, 'Placed');
INSERT INTO placement_records VALUES (3, '20524UCSC003', 'Karthik Raja', 'CSE',
8.2, 0, 'Infosys', 6.5, 'Placed');
INSERT INTO placement_records VALUES (4, '20524UCSC004', 'Meena Priya', 'CSE',
7.6, 0, 'Wipro', 4.5, 'Placed');
INSERT INTO placement_records VALUES (5, '20524UCSC005', 'Naveen Kumar',
'CSE', 6.8, 1, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (6, '20524UIT001', 'Pooja Sri', 'IT', 8.9, 0,
'Accenture', 8.0, 'Placed');
INSERT INTO placement_records VALUES (7, '20524UIT002', 'Rahul Dev', 'IT', 8.4, 0,
'Cognizant', 6.75, 'Placed');
```

```
INSERT INTO placement_records VALUES (8, '20524UIT003', 'Sanjay Kumar', 'IT', 7.9, 0, 'TCS', 4.0, 'Placed');
INSERT INTO placement_records VALUES (9, '20524UIT004', 'Tamil Selvi', 'IT', 7.2, 0, 'Infosys', 5.0, 'Placed');
INSERT INTO placement_records VALUES (10, '20524UIT005', 'Uma Maheswari', 'IT', 6.5, 2, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (11, '20524UEC001', 'Vignesh Raj', 'ECE', 8.8, 0, 'Capgemini', 7.5, 'Placed');
INSERT INTO placement_records VALUES (12, '20524UEC002', 'Yamini Devi', 'ECE', 8.1, 0, 'Wipro', 5.5, 'Placed');
INSERT INTO placement_records VALUES (13, '20524UEC003', 'Arun Prasad', 'ECE', 7.7, 0, 'Cognizant', 4.75, 'Placed');
INSERT INTO placement_records VALUES (14, '20524UEC004', 'Bharathi Raja', 'ECE', 7.0, 1, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (15, '20524UEC005', 'Chitra Devi', 'ECE', 8.0, 0, 'Infosys', 6.0, 'Placed');
INSERT INTO placement_records VALUES (16, '20524UEE001', 'Deepak Kumar', 'EEE', 8.6, 0, 'Zoho', 8.5, 'Placed');
INSERT INTO placement_records VALUES (17, '20524UEE002', 'Eswari Priya', 'EEE', 7.8, 0, 'Accenture', 6.0, 'Placed');
INSERT INTO placement_records VALUES (18, '20524UEE003', 'Gowtham Raj', 'EEE', 7.3, 0, 'TCS', 4.5, 'Placed');
INSERT INTO placement_records VALUES (19, '20524UEE004', 'Harini Sri', 'EEE', 6.9, 1, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (20, '20524UEE005', 'Ilakkiya Devi', 'EEE', 6.2, 3, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (21, '20524UME001', 'Jeeva Kumar', 'MECH', 8.3, 0, 'Capgemini', 5.5, 'Placed');
INSERT INTO placement_records VALUES (22, '20524UME002', 'Kavin Raj', 'MECH', 7.5, 0, 'Wipro', 4.0, 'Placed');
INSERT INTO placement_records VALUES (23, '20524UME003', 'Lavanya Sri', 'MECH', 7.1, 0, 'HCL', 5.0, 'Placed');
INSERT INTO placement_records VALUES (24, '20524UME004', 'Mohan Raj', 'MECH', 6.6, 1, NULL, NULL, 'Not Placed');
INSERT INTO placement_records VALUES (25, '20524UME005', 'Nithya Sri', 'MECH', 6.0, 2, NULL, NULL, 'Not Placed');
```

Records Inserted

A total of 25 student placement records were inserted into the PLACEMENT_RECORDS table.

Output

```
$ sqlite3 placement_db.db
sqlite> SELECT * FROM placement_records;
```

student_id	reg_no	student_name	department	cgpa	backlogs	company	offer_amount	placement_status
1	20524UCSC001	Aarav Kumar	CSE	9.1	0	Zoho	9.5	Placed
2	20524UCSC002	Divya Lakshmi	CSE	8.7	0	TCS	7.0	Placed
3	20524UCSC003	Karthik Raja	CSE	8.2	0	Infosys	6.5	Placed
4	20524UCSC004	Meena Priya	CSE	7.6	0	Wipro	4.5	Placed
5	20524UCSC005	Naveen Kumar	CSE	6.8	1			Not Placed
6	20524UIT001	Pooja Sri	IT	8.9	0	Accenture	8.0	Placed
7	20524UIT002	Rahul Dev	IT	8.4	0	Cognizant	6.75	Placed
8	20524UIT003	Sanjay Kumar	IT	7.9	0	TCS	4.0	Placed
9	20524UIT004	Tamil Selvi	IT	7.2	0	Infosys	5.0	Placed
10	20524UIT005	Uma Maheswari	IT	6.5	2			Not Placed
11	20524UEC001	Vignesh Raj	ECE	8.8	0	Capgemini	7.5	Placed
12	20524UEC002	Yamini Devi	ECE	8.1	0	Wipro	5.5	Placed
13	20524UEC003	Arun Prasad	ECE	7.7	0	Cognizant	4.75	Placed
14	20524UEC004	Bharathi Raja	ECE	7.0	1			Not Placed
15	20524UEC005	Chitra Devi	ECE	8.0	0	Infosys	6.0	Placed
16	20524UEE001	Deepak Kumar	EEE	8.6	0	Zoho	8.5	Placed
17	20524UEE002	Eswari Priya	EEE	7.8	0	Accenture	6.0	Placed
18	20524UEE003	Gowtham Raj	EEE	7.3	0	TCS	4.5	Placed
19	20524UEE004	Harini Sri	EEE	6.9	1			Not Placed
20	20524UEE005	Ilakkiya Devi	EEE	6.2	3			Not Placed
21	20524UME001	Jeeva Kumar	MECH	8.3	0	Capgemini	5.5	Placed
22	20524UME002	Kavin Raj	MECH	7.5	0	Wipro	4.0	Placed
23	20524UME003	Lavanya Sri	MECH	7.1	0	HCL	5.0	Placed
24	20524UME004	Mohan Raj	MECH	6.6	1			Not Placed
25	20524UME005	Nithya Sri	MECH	6.0	2			Not Placed

```
sqlite>
```

SQL Query 1: Display Student Placement Records

SELECT student_id, student_name, department, cgpa, company, offer_amount, placement_status
FROM placement_records
ORDER BY student_id;

Output

```
$ sqlite3 placement_db.db
sqlite> SELECT student_id, student_name, department, cgpa, company, offer_amount, placement_status
FROM placement_records
ORDER BY student_id;
```

student_id	student_name	department	cgpa	company	offer_amount	placement_status
1	Aarav Kumar	CSE	9.1	Zoho	9.5	Placed
2	Divya Lakshmi	CSE	8.7	TCS	7.0	Placed
3	Karthik Raja	CSE	8.2	Infosys	6.5	Placed
4	Meena Priya	CSE	7.6	Wipro	4.5	Placed
5	Naveen Kumar	CSE	6.8			Not Placed
6	Pooja Sri	IT	8.9	Accenture	8.0	Placed
7	Rahul Dev	IT	8.4	Cognizant	6.75	Placed
8	Sanjay Kumar	IT	7.9	TCS	4.0	Placed
9	Tamil Selvi	IT	7.2	Infosys	5.0	Placed
10	Uma Maheswari	IT	6.5			Not Placed
11	Vignesh Raj	ECE	8.8	Capgemini	7.5	Placed
12	Yamini Devi	ECE	8.1	Wipro	5.5	Placed
13	Arun Prasad	ECE	7.7	Cognizant	4.75	Placed
14	Bharathi Raja	ECE	7.0			Not Placed
15	Chitra Devi	ECE	8.0	Infosys	6.0	Placed
16	Deepak Kumar	EEE	8.6	Zoho	8.5	Placed
17	Eswari Priya	EEE	7.8	Accenture	6.0	Placed
18	Gowtham Raj	EEE	7.3	TCS	4.5	Placed
19	Harini Sri	EEE	6.9			Not Placed
20	Ilakkiya Devi	EEE	6.2			Not Placed
21	Jeeva Kumar	MECH	8.3	Capgemini	5.5	Placed
22	Kavin Raj	MECH	7.5	Wipro	4.0	Placed
23	Lavanya Sri	MECH	7.1	HCL	5.0	Placed
24	Mohan Raj	MECH	6.6			Not Placed
25	Nithya Sri	MECH	6.0			Not Placed

```
sqlite>
```

SQL Query 2: Number of Placed Students in Each Department

```
SELECT department, COUNT(*) AS placed_students
FROM placement_records
WHERE placement_status = 'Placed'
GROUP BY department
ORDER BY department;
```

Output

```
sqlite> SELECT department, COUNT(*) AS placed_students
FROM placement_records
WHERE placement_status = 'Placed'
GROUP BY department
ORDER BY department;
```

department	placed_students
CSE	4
ECE	4
EEE	3
IT	4
MECH	3

```
sqlite>
```

SQL Query 3: Average Salary Package Offered by Each Company

```
SELECT company, ROUND(AVG(offer_amount), 2) AS avg_salary
FROM placement_records
WHERE placement_status = 'Placed'
GROUP BY company
ORDER BY company;
```

Output

```
sqlite> SELECT company, ROUND(AVG(offer_amount), 2) AS avg_salary
FROM placement_records
WHERE placement_status = 'Placed'
GROUP BY company
ORDER BY company;
```

company	avg_salary
Accenture	7.0
Capgemini	6.5
Cognizant	5.75
HCL	5.0
Infosys	5.83
TCS	5.17
Wipro	4.67
Zoho	9.0

```
sqlite>
```

SQL Query 4: Students with CGPA of 8 or Above

```
SELECT student_id, student_name, department, cgpa, placement_status
FROM placement_records
WHERE cgpa >= 8
ORDER BY cgpa DESC, student_id;
```

Output

```
sqlite> SELECT student_id, student_name, department, cgpa, placement_status
FROM placement_records
WHERE cgpa >= 8
ORDER BY cgpa DESC, student_id;
```

student_id	student_name	department	cgpa	placement_status
1	Aarav Kumar	CSE	9.1	Placed
6	Pooja Sri	IT	8.9	Placed
11	Vignesh Raj	ECE	8.8	Placed
2	Divya Lakshmi	CSE	8.7	Placed
16	Deepak Kumar	EEE	8.6	Placed
7	Rahul Dev	IT	8.4	Placed
21	Jeeva Kumar	MECH	8.3	Placed
3	Karthik Raja	CSE	8.2	Placed
12	Yamini Devi	ECE	8.1	Placed
15	Chitra Devi	ECE	8.0	Placed

```
sqlite>
```

SQL Query 5: Highest Salary Package Offered in Each Department

```
SELECT department, MAX(offer_amount) AS highest_salary
FROM placement_records
GROUP BY department
ORDER BY department;
```

Output

```
sqlite> SELECT department, MAX(offer_amount) AS highest_salary
FROM placement_records
GROUP BY department
ORDER BY department;
```

department	highest_salary
CSE	9.5
ECE	7.5
EEE	8.5
IT	8.0
MECH	5.5

```
sqlite>
```

PL/SQL Function 1: Check Placement Eligibility

```
CREATE OR REPLACE FUNCTION fn_check_eligibility (
   p_cgpa IN NUMBER,
   p_threshold IN NUMBER)
RETURN VARCHAR2
IS
BEGIN
   IF p_cgpa IS NULL OR p_threshold IS NULL THEN
```

```
      RAISE_APPLICATION_ERROR(-20001, 'CGPA and threshold cannot be NULL.');
   ELSIF p_cgpa < 0 OR p_cgpa > 10 THEN
      RAISE_APPLICATION_ERROR(-20002, 'CGPA must be between 0 and 10.');
   END IF;
   IF p_cgpa >= p_threshold THEN
      RETURN 'Eligible';
   ELSE
      RETURN 'Not Eligible';
   END IF;
END fn_check_eligibility;
/
```

Output (Function created)

(Paste output screenshot here)

Execution

```
SELECT student_id, student_name, cgpa,
   fn_check_eligibility(cgpa, 7.5) AS eligibility
FROM placement_records
WHERE student_id IN (1, 4, 9, 20);
```

Output

(Paste output screenshot here)

PL/SQL Procedure 2: Update Placement Status and Offer Details

```
CREATE OR REPLACE PROCEDURE sp_update_placement (
   p_student_id IN NUMBER,
   p_company IN VARCHAR2,
   p_offer IN NUMBER)
IS
   v_name placement_records.student_name%TYPE;
   v_dept placement_records.department%TYPE;
   v_cgpa placement_records.cgpa%TYPE;
BEGIN
   IF p_student_id IS NULL OR p_company IS NULL OR p_offer IS NULL THEN
      RAISE_APPLICATION_ERROR(-20003, 'Student ID, company and offer amount
      are required.');
   ELSIF p_offer <= 0 THEN
      RAISE_APPLICATION_ERROR(-20004, 'Offer amount must be greater than 0.');
   END IF;
```

```
    SELECT student_name, department, cgpa
    INTO v_name, v_dept, v_cgpa
    FROM placement_records
    WHERE student_id = p_student_id;
    IF fn_check_eligibility(v_cgpa, 6.5) = 'Not Eligible' THEN
       RAISE_APPLICATION_ERROR(-20006, 'Student is not eligible (CGPA below
       6.5).');
    END IF;
    UPDATE placement_records
    SET company = p_company,
       offer_amount = p_offer,
       placement_status = 'Placed'
    WHERE student_id = p_student_id;
    DBMS_OUTPUT.PUT_LINE('Student : ' || v_name || ' (' || v_dept || ')');
    DBMS_OUTPUT.PUT_LINE('CGPA : ' || v_cgpa);
    DBMS_OUTPUT.PUT_LINE('Placement status : Placed');
    DBMS_OUTPUT.PUT_LINE('Company : ' || p_company);
    DBMS_OUTPUT.PUT_LINE('Offer amount (LPA) : ' || p_offer);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
       RAISE_APPLICATION_ERROR(-20005,
          'No student found with the given Student ID.');
END sp_update_placement;
/
```

Output (Procedure created)

(Paste output screenshot here)

Execution

```
SET SERVEROUTPUT ON;
BEGIN
    sp_update_placement(5, 'Wipro', 4.5);
END;
/
ROLLBACK;
```

Output

(Paste output screenshot here)

PL/SQL Procedure 3: Department-wise Placement Report

```
CREATE OR REPLACE PROCEDURE sp_dept_placement_report
IS
   v_count NUMBER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('DEPARTMENT-WISE PLACEMENT REPORT');
   DBMS_OUTPUT.PUT_LINE(RPAD('DEPARTMENT', 12) || RPAD('TOTAL', 7) ||
      RPAD('PLACED', 8) || RPAD('PLACED %', 10) ||
      RPAD('AVG LPA', 9) || RPAD('MAX LPA', 9) ||
      'MIN LPA');
   FOR r IN (SELECT department,
      COUNT(*) AS total_students,
      SUM(CASE WHEN placement_status = 'Placed' THEN 1 ELSE 0 END) AS
      placed_students,
      ROUND(SUM(CASE WHEN placement_status = 'Placed' THEN 1 ELSE 0 END)
         * 100 / COUNT(*), 2) AS placement_pct,
      ROUND(AVG(offer_amount), 2) AS avg_offer,
      MAX(offer_amount) AS max_offer,
      MIN(offer_amount) AS min_offer
      FROM placement_records
      GROUP BY department
      ORDER BY department)
   LOOP
      v_count := v_count + 1;
      DBMS_OUTPUT.PUT_LINE(RPAD(r.department, 12) || RPAD(r.total_students, 7)
      ||
         RPAD(r.placed_students, 8) || RPAD(r.placement_pct, 10) ||
         RPAD(r.avg_offer, 9) || RPAD(r.max_offer, 9) ||
         r.min_offer);
   END LOOP;
   IF v_count = 0 THEN
      RAISE_APPLICATION_ERROR(-20007, 'No data available to generate report.');
   END IF;
END sp_dept_placement_report;
/
```

Output (Procedure created)

(Paste output screenshot here)

Execution

```
BEGIN
   sp_dept_placement_report;
END;
/
```

Output

(Paste output screenshot here)

Exception Handling

Test Case 1 – NULL CGPA

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('Result: ' || fn_check_eligibility(NULL, 7.5));
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

(Paste output screenshot here)

Test Case 2 – Invalid Offer Amount

```
BEGIN
   sp_update_placement(5, 'Wipro', -2);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

(Paste output screenshot here)

Test Case 3 – Record Not Found

```
BEGIN
   sp_update_placement(99, 'Zoho', 6.5);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

(Paste output screenshot here)

Test Case 4 – Student Not Eligible

```
BEGIN
   sp_update_placement(25, 'Zoho', 6.5);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

(Paste output screenshot here)

Test Case 5 – Duplicate Student ID

```
BEGIN
   INSERT INTO placement_records
   VALUES (1, '20524UCSC099', 'Test Student', 'CSE', 7.5, 0, NULL, NULL, 'Not
   Placed');
EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Duplicate student_id not allowed.');
END;
/
```

Output

(Paste output screenshot here)

Dataset Insights

- The dataset contains 25 student records for 5 departments: CSE, IT, ECE, EEE, and MECH (5 students each).
- 18 of the 25 students (72.0%) are placed; CSE, IT, and ECE have 4 placed students each, while EEE and MECH have 3 each.
- Zoho has the highest average package (9.00 LPA), followed by Accenture (7.00 LPA); Wipro has the lowest (4.67 LPA). The overall average package is 6.03 LPA.
- 10 students have a CGPA of 8 or above, and all 10 of them are placed.
- All 18 placed students have no backlogs, while all 7 students who are not placed have at least one backlog and a CGPA of 7.0 or below.
- The highest single offer is 9.5 LPA from Zoho to Aarav Kumar (CSE).
- The eligibility function with a threshold of 7.5 classifies 15 students as Eligible and 10 as Not Eligible.

Result

Thus, the Campus Placement and Recruitment Management system was successfully implemented using SQL and PL/SQL. The PLACEMENT_RECORDS table was created, student records were inserted, SQL queries were executed successfully, PL/SQL functions and procedures were implemented, and exception handling was tested successfully.